



MANUAL DE USUARIO DEL SISTEMA DE GESTIÓN DE CURSOS COMPLEMENTARIOS “GESTIONYTICS”

JHEFER STIVEN PALECHOR PIAMBA

ANÁLISIS Y DESARROLLO DE SOFTWARE (FICHA 3071104)

MOÍSES VARGAS

INSTRUCTOR

CENTRO AGROPECUARIO

SENA

29 DE MAYO DE 2026

Derechos de Autor

El presente proyecto de software fue desarrollado por el aprendiz Jhefer Stiven Palechor, como parte del proceso de formación en el programa Análisis y Desarrollo de Software, ficha No. 3071104, del Servicio Nacional de Aprendizaje (SENA), Centro Agropecuario, Regional Cauca.

El proyecto fue elaborado bajo la orientación del instructor Moisés Vargas, empleando el stack tecnológico MERN (MongoDB, Express.js, React.js y Node.js), e integrando el diseño de interfaces, el desarrollo del código fuente, el modelado de base de datos en MongoDB y la construcción de la arquitectura de la aplicación.

Tabla 1

Historial de Versiones del Sistema Gestionytics

Versión	Fecha	Descripción	Autores
Versión	Fecha	Descripción	Autores
1.0	Mayo 2026	Versión inicial del software – Sistema Gestionytics	Jhefer Stiven Palechor

Nota. Los roles determinan las versiones del sistema Gestionytics.

1. Introducción

El presente manual técnico describe la arquitectura, los componentes y la estructura del sistema de gestión de cursos complementarios del SENA Centro Agropecuario, Regional Cauca, denominado Gestionytics. Su propósito es documentar de forma detallada las decisiones tecnológicas adoptadas durante el desarrollo, con el fin de facilitar el mantenimiento, la escalabilidad y la comprensión del sistema por parte de futuros desarrolladores o del equipo técnico de la institución.

El sistema fue construido sobre la arquitectura MERN (MongoDB, Express.js, React.js y Node.js), una pila tecnológica moderna orientada al desarrollo de aplicaciones web de alto rendimiento. La aplicación permite crear y administrar las ofertas de cursos complementarios a través de un flujo de aprobación entre instructores, coordinadores y funcionarios.

2. Objetivos

2.1 Objetivo General

Desarrollar un sistema de información web que automatice el proceso de creación y gestión de cursos complementarios del SENA Centro Agropecuario, optimizando los tiempos de registro, organización y administración de la oferta académica.

2.2 Objetivos Específicos

- Diseñar un modelo de base de datos en MongoDB que permita almacenar y organizar eficientemente la información de usuarios, ofertas, inscripciones y solicitudes.
- Implementar una API REST con Node.js y Express.js que gestione toda la lógica de negocio del sistema de manera segura y escalable.
- Desarrollar una interfaz de usuario responsiva con React.js que permita a los diferentes roles operar el sistema de forma intuitiva.
- Garantizar la seguridad de la aplicación mediante autenticación JWT, control de roles y encriptación de contraseñas.

3. Planteamiento del Problema

Actualmente, la creación de cursos complementarios se gestiona, donde los instructores deben diligenciar múltiples formularios extensos para enviar la solicitud de creación del curso. Este proceso presenta diversas dificultades:

- Los formularios son extensos y exigen el ingreso detallado de gran cantidad de información.
- El tiempo de diligenciamiento es elevado, generando carga de trabajo adicional a los instructores.
- El alto volumen de solicitudes simultáneas ralentiza el proceso de revisión por parte de los funcionarios.
- La validación manual de la información retrasa la aprobación o rechazo de los cursos.
- Pueden presentarse inconsistencias o errores en los datos ingresados, lo que obliga a reprocesar solicitudes.

Como consecuencia, el proceso de creación y aprobación de cursos complementarios se vuelve demorado e ineficiente, generando retrasos en la programación académica del centro.

4. Justificación

El desarrollo de Gestionytics surge como respuesta directa a las dificultades identificadas en el proceso actual de gestión de cursos complementarios. La implementación de un sistema basado en la arquitectura MERN permitirá optimizar y automatizar dicho proceso, aportando los siguientes beneficios:

- Reducción significativa en los tiempos de diligenciamiento de solicitudes.
- Disminución de errores e inconsistencias en la información registrada.
- Mejora en los tiempos de revisión, validación y aprobación de ofertas.
- Mayor eficiencia administrativa para instructores, coordinadores y funcionarios.
- Trazabilidad completa del estado de cada oferta a lo largo de todo su ciclo de vida.

El uso de tecnologías modernas como MongoDB, Express.js, React.js y Node.js garantiza un sistema dinámico, seguro y adaptable a las necesidades que se presenten en el Centro Agropecuario.

5. Plataforma de Desarrollo

5.1 Arquitectura del Sistema

El sistema está construido sobre una arquitectura Cliente–Servidor de tres capas, comunicadas mediante protocolo HTTP a través de una API REST:

Tabla 2

Arquitectura del Sistema Gestionytics por Capa Tecnológica

Capa	Tecnología / Descripción
Cliente	Aplicación web desarrollada en React.js con Vite
Servidor	API REST desarrollada en Node.js y Express.js
Base de Datos	MongoDB con Mongoose para el modelado de datos
Comunicación	HTTP mediante la librería Axios
Autenticación	JSON Web Token (JWT) con encriptación bcryptjs

Nota. Las capas se comunican mediante protocolo HTTP a través de la API REST.

5.2 Estructura del Proyecto

El repositorio del proyecto se divide en dos carpetas principales:

- backend → Contiene toda la lógica del servidor, la API REST y la conexión a la base de datos.

- frontend → Contiene la interfaz gráfica de usuario desarrollada con React.js y Vite.

5.2.1 Estructura del Backend

Organización de carpetas y archivos del servidor:

- config/ → Configuración de la conexión a MongoDB y variables de entorno.
- controllers/ → Lógica de negocio: crear oferta, aprobar solicitud, gestionar inscripciones, etc.
- middleware/ → Autenticación JWT y verificación de roles (authMiddleware).
- models/ → Esquemas de MongoDB definidos con Mongoose (Usuario, Oferta, Solicitud, etc.).
- routes/ → Definición de rutas de la API REST agrupadas por módulo.
- uploads/ → Carpeta donde se almacenan los archivos PDF cargados al sistema.
- utils/ → Funciones auxiliares como generarFichaPDF.js.
- index.js → Archivo principal del servidor, punto de entrada de la aplicación.
- .env → Variables de entorno (no incluido en el repositorio por seguridad).

5.2.2 Dependencias del Backend

Tabla 3

Dependencias del Backend y Sus Funciones en el Sistema

Dependencia	Versión	Función en el Sistema
-------------	---------	-----------------------

express	^5.2.1	Motor principal del servidor y definición de rutas API REST
mongoose	^9.2.1	Modelado de datos y conexión con MongoDB
jsonwebtoken	^9.0.3	Generación y verificación de tokens JWT para autenticación
bcryptjs	^3.0.3	Encriptación de contraseñas de usuario
cors	^2.8.6	Configuración de permisos de acceso entre dominios (CORS)
dotenv	^17.3.1	Manejo de variables de entorno y configuración sensible
multer	^2.0.2	Carga de archivos PDF adjuntos a las ofertas
pdfkit	^0.17.2	Generación de documentos PDF (reportes y fichas)
xlsx	^0.18.5	Exportación de datos a Excel para reportes académicos
nodemailer	^8.0.1	Envío de notificaciones por correo electrónico
nodemon	^3.1.11	Reinicio automático del servidor en desarrollo

Nota. Las versiones corresponden a las instaladas durante el desarrollo del proyecto en 2026.

5.2.3 Estructura del Frontend

Organización de carpetas dentro de la carpeta frontend/src:

- context/ → Manejo del estado global de la aplicación mediante Context API.
- hooks/ → Hooks personalizados reutilizables en los componentes.
- pages/ → Vistas principales: Login, Registro, Crear Oferta, Mis Ofertas, etc.
- services/ → Módulo de comunicación con el backend (api.js usando Axios).
- App.jsx → Componente raíz con la definición de rutas del sistema.
- main.jsx → Punto de entrada de la aplicación React.

5.2.4 Dependencias del Frontend

Tabla 4

Dependencias del Frontend y Sus Funciones en el Sistema

Dependencia	Versión	Función en el Sistema
react	^18.x	Librería principal para construcción de interfaces de usuario
vite	^5.x	Empaquetador y servidor de desarrollo ultra rápido
axios	^1.x	Cliente HTTP para comunicación con la API REST
react-router-dom	^6.x	Enrutamiento del lado del cliente (SPA)
context api	Nativa	Manejo de estado global de la aplicación

Nota. Todas las dependencias están disponibles en el registro público de npm.

6. Seguridad Implementada

El sistema implementa múltiples capas de seguridad para proteger la información de los usuarios y garantizar el acceso controlado a cada funcionalidad:

- Autenticación mediante JSON Web Token (JWT): cada solicitud protegida requiere un token válido en el header Authorization.
- Encriptación de contraseñas con bcryptjs: las contraseñas nunca se almacenan en texto plano en la base de datos.
- Middleware de protección de rutas: todas las rutas privadas verifican la validez del token antes de ejecutar la lógica.
- Control de acceso por roles: los endpoints están restringidos según el rol del usuario (instructor, coordinador, funcionario).
- Validación de archivos: Multer restringe la carga únicamente a archivos con extensión PDF y con un límite máximo de 10 MB.
- Variables de entorno: los datos sensibles como el JWT_SECRET y la cadena de conexión a MongoDB se gestionan mediante archivo .env.

7. Instalación y Despliegue

7.1 Requisitos Previos

- Node.js versión 18 o superior.
- MongoDB versión 6.0 o superior (local o Atlas).
- npm versión 9 o superior.
- Git para clonar el repositorio.

7.2 Pasos de Instalación

Backend

- Ingresar a la carpeta del servidor: `cd backend`
- Instalar dependencias: `npm install`
- Crear el archivo `.env` con las variables: `PORT=4000`, `MONGO_URI=<cadena-de-conexion>`, `JWT_SECRET=<clave-secreta>`
- Iniciar el servidor en desarrollo: `npm run dev`
- El servidor estará disponible en: `http://localhost:4000`

Frontend

- Ingresar a la carpeta del cliente: `cd frontend`
- Instalar dependencias: `npm install`
- Verificar que la URL base de la API en `services/api.js` apunte a `http://localhost:4000/api`
- Iniciar la aplicación en desarrollo: `npm run dev`

- La aplicación estará disponible en: <http://localhost:5173>

8. Documentación de Endpoints API

Todos los endpoints se consumen desde la URL base `http://localhost:4000/api` en el entorno de desarrollo. Los endpoints marcados como "Autenticado" requieren el header: `Authorization: Bearer {token}`.

Tabla 5

Endpoints de la API REST del Sistema Gestionytics

Método	Endpoint	Descripción	Acceso
POST	<code>/api/auth/login</code>	Autenticar usuario y obtener token JWT	Público
POST	<code>/api/auth/registro</code>	Registrar nuevo instructor o coordinador	Público
POST	<code>/api/ofertas</code>	Crear nueva oferta de curso complementario	Instructor
GET	<code>/api/ofertas/mis-ofertas</code>	Obtener todas las ofertas del instructor autenticado	Instructor
GET	<code>/api/ofertas/{id}</code>	Obtener detalle de una oferta por su identificador	Autenticado
PUT	<code>/api/ofertas/{id}</code>	Actualizar datos de una oferta existente	Instructor
DELETE	<code>/api/ofertas/{id}</code>	Eliminar una oferta (solo en estado borrador)	Instructor
GET	<code>/api/ofertas/{id}/pdf</code>	Descargar ficha PDF generada de la oferta	Autenticado
POST	<code>/api/solicitudes/validacion</code>	Enviar oferta al coordinador para revisión	Instructor

GET	/api/solicitudes/pendientes	Listar solicitudes pendientes de revisión	Coordinador
PUT	/api/solicitudes/{id}/aprobar	Aprobar una solicitud de curso	Coordinador
PUT	/api/solicitudes/{id}/rechazar	Rechazar una solicitud con comentarios	Coordinador
POST	/api/inscripciones/oferta/{codigo}	Inscribir aspirante mediante link público	Público
GET	/api/inscripciones/oferta/{ofertald}	Listar inscritos de una oferta	Autenticado
GET	/api/inscripciones/oferta/{id}/exportar/completo	Exportar lista de inscritos a Excel	Autenticado
POST	/api/ficha/{ofertald}/iniciar-proceso	Iniciar proceso de creación de ficha	Funcionario
GET	/api/ficha/{ofertald}/generar-excel	Generar Excel de aspirantes para matrícula	Funcionario
POST	/api/ficha/{ofertald}/confirmar-matricula	Confirmar matrícula completada	Funcionario
GET	/api/programas-formacion	Obtener catálogo de programas de formación	Público
GET	/api/municipios	Obtener listado de municipios disponibles	Público

Nota. Los endpoints marcados como Autenticado requieren token JWT en el encabezado de la solicitud.

8.1 Estados del Sistema

El sistema gestiona el ciclo de vida de cada oferta mediante los siguientes estados:

Tabla 6

Estados del Ciclo de Vida de una Oferta en Gestionytics

Código	Nombre	Descripción
pendiente	Pendiente	Oferta recién creada, lista para enviar al coordinador
rechazada	Rechazada	El coordinador rechazó la solicitud con observaciones
aprobada	Aprobada	El coordinador aprobó y remitió al funcionario
lista_espera	Lista de Espera	Aprobada, esperando revisión del funcionario SENA
ficha_proceso_creacion	Ficha en Proceso	Funcionario revisando documentos y creando ficha
ficha_creada	Ficha Creada	Excel enviado al instructor para validación de aspirantes
validacion_instructor	Validación Instructor	Instructor validando la lista de aspirantes matriculados
matriculados	Ficha Matriculada	Proceso completo, ficha matriculada en Sofia Plus

Nota. Los estados determinan las acciones disponibles para cada rol en el sistema.

9. Modelo de Datos

El sistema utiliza MongoDB como motor de base de datos NoSQL orientado a documentos. A continuación se describen las colecciones principales con sus campos, tipos de dato y restricciones.

9.1 Colección: Usuario

Tabla 7

Estructura de la Colección Usuario en MongoDB

Campo	Tipo	Descripción	Restricciones
_id	ObjectId	Identificador único del documento	PK (generado por MongoDB)
nombre	String	Nombres del usuario	NOT NULL
apellido	String	Apellidos del usuario	NOT NULL
tipoDocumento	String	Tipo de identificación (CC, TI, CE...)	NOT NULL
numeroDocumento	String	Número de documento de identidad	UNIQUE, NOT NULL
nombreUsuario	String	Nombre de usuario para login	UNIQUE, NOT NULL
correoElectronico	String	Correo electrónico institucional	UNIQUE, NOT NULL
password	String	Contraseña encriptada con bcryptjs	NOT NULL

rol	String	instructor coordinador funcionario admin	NOT NULL
telefono	String	Número de contacto	
isActive	Boolean	Indica si la cuenta está activa	DEFAULT true
coordinador_id	ObjectId	Referencia al coordinador asignado	FK → Coordinador

Nota. Los campos marcados como NOT NULL son obligatorios en el esquema de Mongoose.

9.2 Colección: Oferta

Tabla 8

Estructura de la Colección Oferta en MongoDB

Campo	Tipo	Descripción	Restricciones
_id	ObjectId	Identificador único	PK
idFicha	Number	Número de ficha asignada	UNIQUE, NOT NULL
codigo	String	Código generado automáticamente	UNIQUE
cupo	Number	Cupo máximo de aprendices	NOT NULL, >=0
aprendicesInscritos	Number	Cupos ocupados actualmente	DEFAULT 0
fechaInicio	Date	Fecha de inicio del curso	
fechaFin	Date	Fecha de finalización del curso	
horaInicio	String	Hora de inicio en formato HH:MM	

horaFin	String	Hora de finalización en formato HH:MM	
diasFormacion	String	Días de formación separados por coma	
modalidad_id	ObjectId	Modalidad del curso	FK → Modalidad
tipoOferta_id	ObjectId	Tipo de oferta (abierta/cerrada)	FK → TipoOferta
programaFormacion_id	ObjectId	Programa de formación asociado	FK → ProgramaFormacion
municipio_id	ObjectId	Municipio donde se realiza	FK → Municipio
empresa_id	ObjectId	Empresa solicitante (oferta cerrada)	FK → Empresa
estado_id	ObjectId	Estado actual del proceso	FK → EstadoOferta
creador_id	ObjectId	Instructor que creó la oferta	FK → Usuario
firmaDigital	String	Ruta del PDF de firma digital	
fichaCaracterizacion	String	Ruta del PDF de caracterización	

Nota. Los campos con prefijo _id que referencian otras colecciones son claves foráneas en el modelo relacional de Mongoose.

10. Acuerdos de Niveles de Servicio (ANS)

Los acuerdos de niveles de servicio definen los compromisos de rendimiento y disponibilidad que el sistema debe cumplir para garantizar una operación adecuada.

Tabla 9

Acuerdos de Niveles de Servicio del Sistema Gestionytics

Servicio	Nivel	Métrica	Tiempo	Responsable
Disponibilidad del sistema	99% mensual	Tiempo en línea	24/7 salvo mantenimientos programados	Administrador del sistema
Inicio de sesión	Respuesta rápida	Tiempo de autenticación	≤ 3 segundos	Backend Node.js
Registro de oferta	Procesamiento inmediato	Tiempo de guardado	≤ 3 segundos	API REST
Carga de archivos PDF	Subida segura y validada	Tiempo de carga	≤ 5 segundos por archivo	Backend + Multer
Envío a coordinador	Cambio automático	Actualización en BD	Inmediato	Sistema
Soporte técnico	Atención de incidencias	Tiempo de respuesta	24 – 48 horas hábiles	Equipo técnico

Nota. Los tiempos de respuesta se miden bajo condiciones normales de operación con una conexión de banda ancha estable.